```r
test1 <- c(1.5, 0.7, 45.6)
# Explanation: c() stands for 'combine'. It creates a vector of the three numbers.
# b. Create vector y1 from 1 to 10
y1 <- 1:10
# Explanation: The colon operator (:) generates a sequence of integers.
# c. Create logical vector y2
y2 <- y1 > 5
# Explanation: This evaluates each element of y1. If the value is > 5, it returns TRUE.
# d. Count elements larger than 5
sum(y2)
# Explanation: In R, TRUE equals 1 and FALSE equals 0. Summing a logical vector tells you how
many TRUEs exist.

# b. Vector y (0, 5, 10 ... 500)
y <- seq(from = 0, to = 500, by = 5)
# c. Vector z1 (1,1,1, 2,2,2 ... 50,50,50)
z1 <- rep(1:50, each = 3)
# d. Vector z2 (1, 2,2, 3,3,3 ... 10)
z2 <- rep(1:10, times = 1:10)
# e. Vector z3 (1, 2,2, 3, 4,4, 5, 6,6 ... 50,50)
z3 <- rep(1:50, times = rep(c(1, 2), 25))
# Explanation: We use a repeating pattern (1, 2) to tell R to repeat the odd numbers once and the
even numbers twice.

# Define the function
get_stats <- function(x) {
  stats <- c(Min = min(x),Median = median(x),Max = max(x),Mean = mean(x),SD = sd(x),Length =
length(x))
  return(stats)
}
# Apply to a vector from 25 to 80
x_vector <- 25:80
get_stats(x_vector)

# Define the function
fun1 <- function(x)
{result <- ifelse(x >= 0, "Non-negative number", "Negative number")
 return(result)
}

solve_quadratic <- function(a, b, c)
{ D <- b^2 - 4*a*c
  if (D > 0)
  { x1 <- (-b + sqrt(D)) / (2 * a)
    x2 <- (-b - sqrt(D)) / (2 * a)
    roots <- c(x1, x2)
    print(paste("Two roots found:", round(x1, 4), "and", round(x2, 4)))
  }
  else if (D == 0)
  { roots <- -b / (2 * a)
    print(paste("One root found:", roots))
  }
  else
  { roots <- NULL
    print("No real roots (D < 0)")
  }
  # Create a plot of the function
  curve(a*x^2 + b*x + c, from = -2, to = 2,
        ylab = "f(x)", xlab = "x", col = "blue", lwd = 2)
```

```
  abline(h = 0, lty = 2) # Add a horizontal line at y=0
  return(roots)
}
solve_quadratic(a =-8, b =6, c =4)

library(reshape2)
data(tips)
# Create the scatterplot
plot(tips$total_bill, tips$tip,
     xlab = "total_bill", ylab = "tip",
     main = "Tip vs Total Bill", pch = 19, col = "gray")
# Add a linear regression line (y ~ x)
abline(lm(tip ~ total_bill, data = tips), col = "blue", lwd = 2)
# Add a horizontal reference line for the average tip
abline(h = mean(tips$tip), col = "red", lty = 2, lwd = 2)
# Add a legend to explain the lines
legend("topleft", legend = c("regression line", "average tip"),
       col = c("blue", "red"), lty = c(1, 2), lwd = 2)

# Create the boxplot
boxplot(tip ~ day, data = tips,
        xlab = "tips$day", ylab = "tips$tip",
        main = "Tips by Day of the Week",
        col = "lightblue")

# Define the x-axis range from -2pi to 2pi
x <- seq(-2 * pi, 2 * pi, length.out = 1000)
# Plot the Sine function first to set up the window
plot(x, sin(x), type = "l", lty = 1, col = "black",
     ylim = c(-1.5, 1.5), ylab = "y", main = "Trigonometric Functions")
lines(x, cos(x), lty = 2, col = "blue")
lines(x, tan(x), lty = 3, col = "red")
legend("bottomleft", legend = c("sin", "cos", "tan"),
       lty = 1:3, col = c("black", "blue", "red"))
# Add a horizontal line at 0 for reference
abline(h = 0, col = "gray", lty = "dotted")

library(reshape2)
library(dplyr)
data(tips)

tips.tbl <- as_tibble(tips)
#Subset sub1 (Male AND size >= 3)
sub1 <- filter(tips.tbl, sex == "Male" & size >= 3)
#Subset sub2 (Male OR size >= 3)
sub2 <- filter(tips.tbl, sex == "Male" | size >= 3)
#Subset sub3 (Remove smoker)
sub3 <- select(sub2, -smoker)
#Sort by time and decreasing size
sort1 <- arrange(sub3, time, desc(size))
#Average tip by gender
avg_tip_gender <- sort1 %>%
  group_by(sex) %>%
  summarise(avg_tip = mean(tip))
#Using the Pipe (Steps a-d in one block)
result_pipe <- tips.tbl %>%
  filter(sex == "Male" | size >= 3) %>%
  select(-smoker) %>%
  arrange(time, desc(size)) %>%
```

```
  group_by(sex) %>%
  summarise(avg_tip = mean(tip))
#Average tip by day
avg_tip_day <- tips.tbl %>%
  group_by(day) %>%
  summarise(avg_tip = mean(tip))
#Average tip by gender and day
avg_tip_both <- tips.tbl %>%
  group_by(sex, day) %>%
  summarise(avg_tip = mean(tip))

#Descriptive statistics for total_bill by gender
tips.tbl %>%
  group_by(sex) %>%
  summarise(mean = mean(total_bill),median = median(total_bill),n = n(),stdev = sd(total_bill))

VISUAL <- function(DFR, CONT, CAT, COL1) {
  par(mfrow = c(1, 2))
  # Boxplot
  boxplot(DFR[[CONT]] ~ DFR[[CAT]],
          col = COL1,
          main = paste(CONT, "by", CAT),
          xlab = CAT, ylab = CONT)
  # Histogram with Density Overlay
  hist(DFR[[CONT]], prob = TRUE, col = COL1,
       main = paste("Histogram of", CONT),
       xlab = CONT)
  lines(density(DFR[[CONT]]), lwd = 2)
  par(mfrow = c(1, 1))
}
VISUAL(DFR = tips, CONT = "tip", CAT = "day", COL1 = 3)

summary(airquality)
nrow(airquality)
# 3. Count missing values in Ozone
sum(is.na(airquality$Ozone))
# 4. Create subset with only complete cases
air_complete <- airquality[complete.cases(airquality), ]
# Explanation: complete.cases returns TRUE only for rows with no NAs in any column.
# 5. Count rows in air_complete
nrow(air_complete)

n <- 6 i <- 1
while(i < n)
{print(paste("nr", i))
  i <- i + 1
}

n <- 5 sum_val <- 0
i <- 1
while(i <= n)
{ sum_val <- sum_val + i
  i <- i + 1
}
print(sum_val)

i <- 1
while(i <= 5)
{row_vec <- c()
```

```
  j <- 1
  while(j <= i)
  {row_vec <- c(row_vec, i)
    j <- j + 1
  }
  print(row_vec)
  i <- i + 1
}

library(tidyr)
library(dplyr)

billboard_date <- billboard %>%
  mutate(
    date_temp = as.Date(date.entered),
    Year   = as.numeric(format(date_temp, "%Y")),
    Month  = as.numeric(format(date_temp, "%m")),
    Day_nr = as.numeric(format(date_temp, "%d"))
  ) %>%
  select(artist, wk1, wk2, wk3, wk4, Year, Month, Day_nr)

b_billboard <- billboard_date %>%
  pivot_longer(
    cols = c(wk1, wk2, wk3, wk4),
    names_to = "Week",
    values_to = "Rank"
  )
# Compare dimensions
dim(billboard_date)
dim(b_billboard)
# Pivot wider to create separate estimate and moe columns for income/rent
us_rent_wide <- us_rent_income %>%
  pivot_wider(names_from = variable,
              values_from = c(estimate, moe))

library(qcc)
data(boiler)
x_vals <- boiler$t1
# a. Create total data frame
mr <- c(NA, abs(diff(x_vals)))
total <- data.frame(time = 1:25, x = x_vals, mr = mr)
# b. Compute averages for first 20 points
mr_bar <- mean(total$mr[2:20], na.rm = TRUE)
x_bar <- mean(total$x[1:20])
# c. Control Limits
# Moving Range Limits
LCL_mr <- 0 * mr_bar
UCL_mr <- 3.267 * mr_bar
# Individual Limits
LCL_x <- x_bar - 2.66 * mr_bar
UCL_x <- x_bar + 2.66 * mr_bar
# d. Plotting
par(mfrow = c(1, 2))
# Individual Chart
plot(total$time, total$x, type="b", main="Individual Chart")
abline(h=c(x_bar, LCL_x, UCL_x), col=c("green", "red", "red"), lty=c(1,2,2))
# Moving Range Chart
plot(total$time, total$mr, type="b", main="Moving Range Chart")
abline(h=c(mr_bar, LCL_mr, UCL_mr), col=c("green", "red", "red"), lty=c(1,2,2))
```

```r
#Generate noisy data
x <- seq(1, 3, length.out = 100)
y <- jitter(x, factor = 50)
#Create generalized function
plot_rolling <- function(y, consec = 5) {
  # Calculate rolling mean
  n <- length(y)
  roll_mean <- sapply(1:(n - consec + 1), function(i) {
    mean(y[i:(i + consec - 1)])
  })
  #Plot original and add rolling mean line
  plot(y, pch = 19, col = "gray", main = paste("Rolling Average (n =", consec, ")"))
  lines((consec:n) - (consec/2), roll_mean, col = "blue", lwd = 2)
}
#Apply with 10 points
plot_rolling(y, consec = 10)

# chol <- read.table("chol.txt", header = TRUE)
#Create the 'group' variable
chol$group <- ifelse(chol$SMOKE == 0, "nonsmoke", "smoke")
#Grouped boxplot
boxplot(CHOL ~ group, data = chol,
        main = "Cholesterol Levels by Smoking Status",
        xlab = "Group", ylab = "Cholesterol",
        col = c("lightblue", "orange"))
par(mfrow = c(1, 2))
# Histogram for Nonsmokers
hist(chol$CHOL[chol$group == "nonsmoke"],
     main = "Nonsmokers", xlab = "CHOL", col = "lightblue", ylim = c(0, 20))
# Histogram for Smokers
hist(chol$CHOL[chol$group == "smoke"],
     main = "Smokers", xlab = "CHOL", col = "orange", ylim = c(0, 20))
par(mfrow = c(1, 1))

# Perform the t-test
t_result <- t.test(CHOL ~ group, data = chol)
print(t_result)

obs <- matrix(c(61, 28, 7,
                68, 23, 13,
                58, 40, 12,
                53, 38, 16),
              nrow = 4, byrow = TRUE)
rownames(obs) <- c("A", "B", "C", "D")
colnames(obs) <- c("0", "1", "2")
# Convert to a table object
operations_table <- as.table(obs)
print(operations_table)
# Check for independence
chisq_test_ops <- chisq.test(operations_table)
print(chisq_test_ops)
mosaicplot(operations_table, main = "Mosaic Plot", color = TRUE)

library(reshape2)
data(tips)
# Create a contingency table
tips_table <- table(tips$day, tips$sex)
print(tips_table)
# Check for independence
```

```
chisq_test_tips <- chisq.test(tips_table)
print(chisq_test_tips)
mosaicplot(tips_table, main = "Mosaic Plot of Day vs Gender", color = TRUE)

library(reshape2)
data(tips)
model <- lm(tip ~ total_bill, data = tips)
summary(model)
# Plot the data
plot(tips$total_bill, tips$tip,
     pch = 19, col = "gray",
     xlab = "Total Bill ($)", ylab = "Tip ($)",
     main = "Regression: Tip vs Total Bill")
abline(model, col = "blue", lwd = 2)

par(mfrow = c(2, 2))
plot(model)
par(mfrow = c(1, 1))
# 1. Identify the largest tips
large_tips <- tips[order(tips$tip, decreasing = TRUE), ]
head(large_tips, 3)
# 2. Identify points with high Cook's Distance
cooks_d <- cooks.distance(model)
influential_obs <- which(cooks_d > (4 / nrow(tips)))
tips[influential_obs, ]


# --- Chapter 11 ---
library(ggplot2)
library(Hmisc)
library(grid)
# - Exercise 1 -
library(reshape)
names(tips)
head(tips)
##Boxplot of tip by sex of bill payer
tip1 <- ggplot(tips, aes(x=sex,y=tip)) +
  geom_boxplot(colour="black", fill= "purple")
tip1 + labs(title= "box plot of tip by sex of bill payer")
## Matrix of panels by using faceting variable day of the week
tip2 <- tip1 + facet_grid(. ~ day) + labs(title = "faceting on day of the week")
tip2
##Scatter plot of tips by total_bill, grouping by smoker
tip3 <- ggplot(tips, aes(total_bill, tip, colour = smoker)) +
  geom_point(size = 4) +
  scale_color_manual(name="smoker", values=c("No" = "blue","Yes" = "green"))
tip3 + labs(title = "scatter plot of tip by bill, coloured by smoke")
##Add two regression lines
tip4 <- tip3 + geom_smooth(method = "lm", se = FALSE, linewidth = 2) +
  labs(title = "scatter plot of tip by bill, coloured by smoke")
tip4
#add one regression line for the total group
tip4b <- tip4 + geom_smooth(method = "lm", se = FALSE,colour="black", linewidth=3)
tip4b

## Question 5
hist1 <- ggplot(tips, aes(tip)) +
  geom_histogram(bins = 25, colour = "green", fill = "green") + theme_bw()
boxplot1 <- ggplot(tips, aes(sex, tip)) +
```

```
  geom_boxplot(fill = "pink") + theme_bw()
scatter1 <- ggplot(tips, aes(total_bill, tip, colour = smoker)) +
  geom_point(size = 3) + theme_bw()
grid.newpage()
pushViewport(viewport(layout=grid.layout(2,3)))
vplayout <- function(x,y)
{viewport(layout.pos.row=x, layout.pos.col=y)}
print(hist1, vp=vplayout(1,1:3))
print(boxplot1, vp=vplayout(2,3))
print(scatter1, vp=vplayout(2,1:2))
## Question 6
part1 <- ggplot(tips, aes(day, tip)) + geom_point()
part2 <- part1 + stat_summary(fun.data = "mean_cl_boot", size = 1, colour = "blue")
part3 <- part2 + labs(title = "Use of mean and corresponding confidence interval")
part4 <- part3 + theme_bw()
part4

# - Exercise 2 -
library(UsingR)
##Create scatter plot of Initial height (X) versus horizontal distance (Y )
p <- ggplot(galileo, aes(x=init.h, y=h.d))
p1 <- p + geom_point(colour="black",size=3)
##Add regression line
p3 <- p1 + geom_smooth(formula= y~x, method="lm", se=F, aes(colour="linear"))
##Add quadratic line
p4 <- p3 + geom_smooth(formula= y~ x +I(x^2), method="lm", se=F, aes(colour="quadratic"))
##
p5 <- p4 + scale_colour_manual(name="Method", values=c("green","blue")) +
  xlab("initial height") +ylab("horizontal distance")+labs(title="Galileo data")
p5

# - Exercise 3 -
x <- seq(from  =-2*pi, to = 2*pi, by = 0.05)
sin_x <- sin(x)
cos_x <- cos(x)
tan_x <- tan(x)
# Create a data frame
geom <- data.frame(x, sin_x, cos_x, tan_x )
# Create the plot
p <- ggplot(geom, aes(x=x))
p1 <- p + geom_line(aes(y = sin_x),col="red") +
  geom_line(aes(y = cos_x), col="blue") +
  geom_line(aes(y = tan_x), col="green")+ theme_bw()
p1 +  scale_y_continuous(limits = c(-2, +2))
# more advanced
p <- ggplot(geom, aes(x=x, y=sin_x))
p1 <- p + geom_line(aes(colour="sin_x")) +
  geom_line(aes(y = cos_x, colour = "cos_x")) +
  geom_line(aes(y = tan_x, colour = "tan_x"))
p2 <- p1  + scale_y_continuous(limits = c(-2, +2)) + ylab(" " ) +
  labs(title = "Geometric curves") + labs(colour = "Geometric function")
p2

library(shiny)
library(ggplot2)
library(reshape2)
data(tips)

ui <- fluidPage(
```

```
  titlePanel("Exercises 1 & 2: Boxplot and Histogram"),
  sidebarLayout(
    sidebarPanel(
      # Exercise 1: Radio Buttons for Color
      radioButtons("box_col", "Select Boxplot Color:",
                   choices = c("Blue" = "skyblue", "Red" = "tomato", "Green" = "springgreen")),
      hr(),
      # Exercise 2: Slider for Bins
      sliderInput("bins", "Number of bins:", min = 5, max = 30, value = 15)
    ),
    mainPanel(
      plotOutput("boxPlot"),
      plotOutput("histPlot")
    )
  )
)

server <- function(input, output) {
  output$boxPlot <- renderPlot({
    ggplot(tips, aes(x = day, y = total_bill)) +
      geom_boxplot(fill = input$box_col) +
      theme_minimal()
  })

  output$histPlot <- renderPlot({
    ggplot(tips, aes(x = total_bill)) +
      geom_histogram(bins = input$bins, fill = "gray", color = "white") +
      theme_minimal()
  })
}

shinyApp(ui, server)

ui <- fluidPage(
  checkboxInput("show_sex", "Color by Sex", value = FALSE),
  plotOutput("scatterPlot")
)

server <- function(input, output) {
  output$scatterPlot <- renderPlot({
    p <- ggplot(tips, aes(x = total_bill, y = tip))

    if (input$show_sex) {
      p <- p + geom_point(aes(color = sex))
    } else {
      p <- p + geom_point()
    }
    p + theme_minimal()
  })
}

shinyApp(ui, server)

ui <- fluidPage(
  selectInput("smoke_filter", "Filter by Smoker:",
              choices = c("All", "Yes", "No")),
  plotOutput("smokePlot")
)
```

```
server <- function(input, output) {
  output$smokePlot <- renderPlot({
    # Logic for "All"
    plot_data <- if (input$smoke_filter == "All") {
      tips
    } else {
      subset(tips, smoker == input$smoke_filter)
    }

    ggplot(plot_data, aes(x = total_bill, y = tip)) +
      geom_point(color = "darkblue") +
      theme_minimal()
  })
}

shinyApp(ui, server)

ui <- fluidPage(
  titlePanel("Stacked Plots with Dynamic Faceting"),
  sidebarLayout(
    sidebarPanel(
      selectInput("facet_var", "Select Faceting Variable:",
                  choices = c("sex", "smoker", "time"))
    ),
    mainPanel(
      # Stacked vertically in mainPanel
      plotOutput("facetBox"),
      plotOutput("facetScatter")
    )
  )
)

server <- function(input, output) {
  # We create the dynamic formula
  facet_formula <- reactive({
    as.formula(paste(". ~", input$facet_var))
  })

  output$facetBox <- renderPlot({
    ggplot(tips, aes(x = day, y = total_bill)) +
      geom_boxplot(fill = "lightblue") +
      facet_grid(facet_formula()) +
      theme_bw()
  })

  output$facetScatter <- renderPlot({
    ggplot(tips, aes(x = total_bill, y = tip)) +
      geom_point() +
      facet_grid(facet_formula()) +
      theme_bw()
  })
}

shinyApp(ui, server)

#Exam
#Define x and the first value of the series
x <- pi / 6
series_sin <- c(x)
```

```
#Compute the 2nd and 3rd values
val2 <- x - (x^3 / factorial(3))
val3 <- x - (x^3 / factorial(3)) + (x^5 / factorial(5))
# Combine into the vector
series_sin <- c(x, val2, val3)
print(series_sin)

fun_sin <- function(n = 5, x = pi / 2) {
  approx_sin <- 0
  for (k in 1:n) {
    exponent <- 2 * k - 1
    term <- ((-1)^(k - 1)) * (x^exponent / factorial(exponent))
    approx_sin <- approx_sin + term
  }
  return (list(n = n, x = x, result = approx_sin))
}
result1 <- fun_sin(x = pi / 6, n = 3)
print(result1)
result2 <- fun_sin(x = pi / 4, n = 6)
print(result2)

library(readxl)
library(dplyr)
library(ggplot2)
library(ggExtra)

# Import the data
df <- read_excel("Business.xlsx")
sum(is.na(df$Business_status))
sum(is.na(df$sex))
sub1 <- df %>%
  mutate(Business_status = na_if(Business_status, 4))
sub2 <- sub1 %>%
  filter(neoconsc > 30)
ggplot(sub2, aes(x = neoopen, y = neoconsc, color = factor(Business_status))) +
  geom_point() +
  labs(x = "Openness to Experience", y = "Conscientiousness", color = "Business Status")

df %>%
  mutate(Business_status = na_if(Business_status, 4)) %>%
  filter(neoconsc > 30) %>%
  ggplot(aes(x = neoopen, y = neoconsc, color = factor(Business_status))) +
  geom_point()

plot1 <- sub2 %>%
  remove_missing(vars = "Business_status") %>%
  ggplot(aes(x = neoopen, y = neoconsc, color = factor(Business_status))) +
  geom_point() +
  labs(title = "Cleaned Plot")
# Explanation: remove_missing ensures that points with NA in Business_status don't appear in the
legend or plot.

ggMarginal(plot1, type = "histogram", fill = "lightgray")
# Explanation: ggMarginal takes a ggplot object and adds marginal distributions (histograms in
this case).

library(readxl)
library(dplyr)
library(ggplot2)
```

```
boston2 <- read_excel("boston2.xlsx")
summary1 <- boston2 %>%
  group_by(roomfact) %>%
  summarise(
    avg_international = mean(international, na.rm = TRUE),
    n_districts = n()
  )

print(summary1)
boston_center2 <- boston2 %>%
  left_join(summary1, by = "roomfact") %>%
  mutate(diff = international - avg_international) %>%
  dplyr::select(roomfact, average = avg_international, international, diff, crimcat) %>%
  arrange(diff)
head(boston_center2)

#Create boxplot
ggplot(boston_center2, aes(x = roomfact, y = diff, fill = roomfact)) +
  geom_boxplot() +
  geom_hline(yintercept = 0, color = "green", linetype = "dashed", size = 1) +
  labs(title = "Deviation from Average International Proportion by House Type",
       y = "Difference (Observed - Average)",
       x = "Room Category") +
  theme_minimal()
#Define the regression function
regres.f <- function(resp, expl) {
  model <- lm(resp ~ expl)
  mod_sum <- summary(model)
  intercept <- mod_sum$coefficients[1, 1]
  slope <- mod_sum$coefficients[2, 1]
  r_squared <- mod_sum$r.squared
  return(c(Intercept = intercept, Slope = slope, R2 = r_squared))
}

regres.f(boston2$dis, boston2$zn)
# 5. Apply function for every level of roomfact
df1 <- boston2 %>%
  group_by(roomfact) %>%
  summarise(
    results = list(regres.f(dis, zn))
  ) %>%
  tidyr::unnest_wider(results)

print(df1)
```